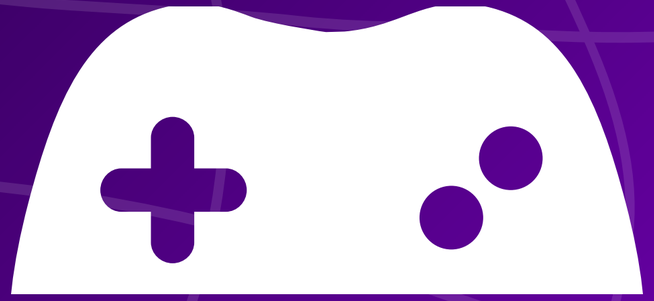
LUCAS MIGUEL VIEIRA DOS SANTOS



GAMESTOCK

Sistema de Gerenciamento de Estoque

IFAL Campus ARAPIRACA

SUMARIO

1. Introdução e demandas
2. Código
3. Estruturas utilizadas
4. Conclusão

INTRODUÇÃO

O GameStock é um sistema de gerenciamento desenvolvido em Python, tendo como cenário uma loja de videogames e jogos.

O projeto surgiu da necessidade de facilitar o controle e a organização dos produtos, permitindo acompanhar o estoque, registrar vendas e consultar informações de maneira prática.

A proposta é oferecer uma solução simples e funcional, capaz de auxiliar nas atividades de gerenciamento do estabelecimento, reduzindo a dependência de controles manuais e centralizando as informações em um único sistema.



Durante a análise do cenário de uma loja de videogames e jogos, percebemos que a falta de um controle organizado dos produtos poderia dificultar diversas atividades do estabelecimento. A dificuldade em acompanhar as quantidades disponíveis no estoque, consultar informações sobre os produtos e registrar as vendas poderia comprometer a organização e tornar o gerenciamento mais demorado.

A partir dessas necessidades, identificamos a importância de uma ferramenta que reunisse essas operações em um único lugar, facilitando o acesso às informações e tornando o controle do estabelecimento mais prático e eficiente.

GAMESTOCK

CÓDIGO

- **1 - Estruturas condicionais (if, elif e else)**

Foram utilizadas para controlar as decisões do programa, permitindo executar diferentes ações de acordo com as opções escolhidas pelo usuário e validar as informações inseridas.

Onde foram utilizadas:

- **main.py:** identificação da opção selecionada no menu principal.
- **produtos.py:** validação de códigos, preços, estoques e confirmações de remoção.
- **vendas.py:** verificação da existência do produto e da disponibilidade de estoque.



CÓDIGO

- **2 - Estruturas de repetição (while e for)**

Foram utilizadas para repetir operações e percorrer os dados armazenados, evitando a necessidade de executar manualmente cada procedimento.

Onde foram utilizadas:

- **while:** mantém o menu principal em execução até que o usuário escolha sair, além de permitir repetir solicitações durante as validações de dados.

- **for:** percorre as listas de produtos e vendas para realizar buscas, consultas e exibições de informações.



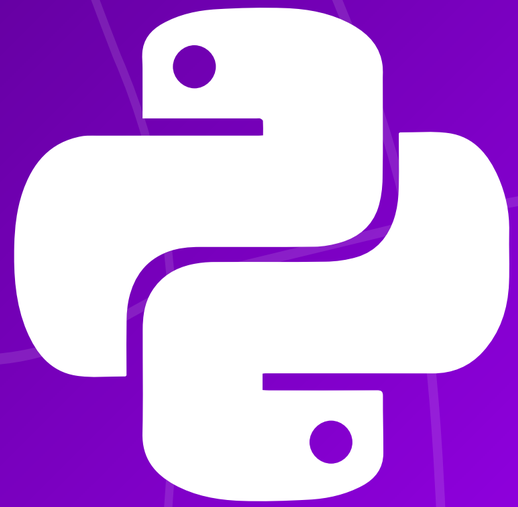
CÓDIGO

- **3 - Subprogramas (funções)**

O código foi dividido em funções, responsáveis por executar tarefas específicas. Essa organização facilita a manutenção, a reutilização e a compreensão do programa.

Exemplos:

- cadastrar_produto()
- listar_produtos()
- consultar_produto()
- remover_produto()
- registrar_venda()
- carregar_dados() e salvar_dados()



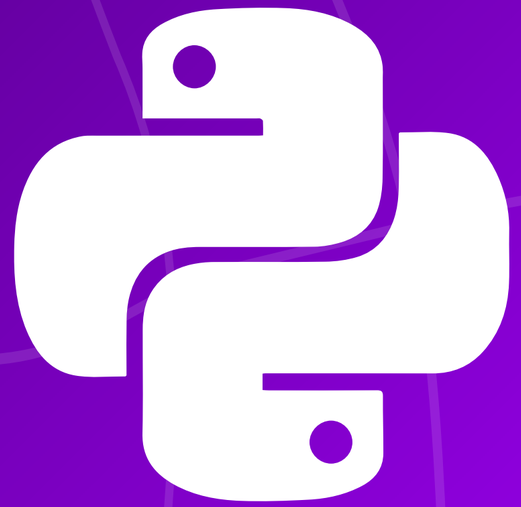
CÓDIGO

- **4 - Listas**

Onde foram utilizadas:

Foram utilizadas para armazenar e organizar conjuntos de informações que podem aumentar ou diminuir durante a execução do programa.

- Lista produtos: armazena os produtos cadastrados.
- Lista vendas: armazena o histórico das vendas realizadas.



CÓDIGO

- **5 - Cadeias de caracteres (strings)**

Foram utilizadas para representar e manipular informações textuais, como códigos, nomes e categorias dos produtos.

Exemplos de utilização:

- input(): recebe informações digitadas pelo usuário.
- .strip(), .lower() e .upper(): auxiliam no tratamento e na padronização dos textos.



CÓDIGO

- **6. Estruturas de dados (dicionários)**

Foram utilizados dicionários para organizar as informações de cada produto e de cada venda, associando valores a identificadores específicos.

Exemplos:

- Produtos: código, nome, categoria, preço e estoque.
- Vendas: código do produto, nome, quantidade, preço unitário e valor total.



CÓDIGO

- **7. Arquivos**

Foi utilizada a biblioteca **json** para realizar o armazenamento e a recuperação dos dados do sistema.

Onde foram utilizadas:

- **carregar_dados():** lê as informações salvas no arquivo dados.json.
- **salvar_dados():** registra os produtos e as vendas no arquivo.



CONCLUSÃO

- Colocar em prática os conceitos de programação estudados, integrando diferentes estruturas e funcionalidades em um único sistema.
- Atendeu aos principais requisitos estabelecidos, criando uma aplicação funcional para o gerenciamento de produtos, controle de estoque e registro de vendas.
- Experiência com organização de código, resolução de problemas e desenvolvimento de software, demonstrando como a programação pode oferecer soluções para necessidades do cotidiano.

OBRIGADO

LUCAS MIGUEL VIEIRA DOS SANTOS